

YODA Group 11 Milestone 1

Joshua Smith

Ebrahim Bhyat

Tlangelani Tembe

May 2026

1 Project Overview

For the HPES Project, our team is extending the baseline StarCore-1 processor by designing a Coordinate Rotation Unit (CRU). The StarCore-1 Base Core Equivalent lacks the specialized logic required for complex trigonometric tasks. Using the CORDIC algorithm, the CRU will act as a hardware accelerator to compute Sine, Cosine, Tangent, and their inverses. This is specifically targeted at offloading satellite altitude determination calculations from the main core, completely avoiding the need for massive memory-mapped Look-Up Tables (LUTs). The CRU interfaces with the datapath using the reserved 1010 opcode, employing a stall-and-handshake methodology to pause the single-cycle PC while the multi-cycle CORDIC iterations complete.

2 Completed Tasks

We have successfully partitioned the initial architecture and verification workload. Our three completed milestones are:

1. **Instruction Set Architecture (ISA) Definition (Josh):** Defined the custom 16-bit instruction format utilizing the reserved 1010 opcode, establishing the register decoding logic required to pass angles to the CRU and read back results.
2. **Golden Measure Generation (Ebrahim):** Simulated the CORDIC algorithm in MATLAB to generate golden measure values for a target angle input. This dataset will be used to automatically verify the accuracy of our Verilog hardware against theoretical expected outputs.
3. **StarCore Datapath Integration (Tlangelani):** Modified and implemented the core routing in `ControlUnit.v` and `Datapath.v`. Created the top-level wrapper (`StarCore1_CRU.v`) to handle the processor stall signals and route the co-processor's result back into the General Purpose Registers.

3 Planned Tasks

Over the next week, we are moving from architectural integration to isolated module implementation and testing. Our three immediate objectives are:

1. **Internal CRU FSM Mapping:** Map the control FSM states for the CRU and define the module interface within Verilog, establishing the foundational architecture for the shift-add iterative math in both Rotation and Vectoring modes.
2. **Isolated CRU Verification :** Code the standalone Verilog Testbench for the CRU module. This will feed the module specific angles and automatically assert correctness against Ebrahim's MATLAB golden measure outputs.
3. **Full System Integration Test:** Develop `StarCore1_CRU_tb.v` to execute a complete StarCore-1 assembly sequence. The test will write an angle to a register, trigger the 1010 opcode, poll the CRU status, and execute Load Word (LW) instructions to retrieve the output values.

Table 1: Coordinate Rotation Unit (CRU) Instruction Set Architecture

Instruction	[15:12]	[11:9]	[8:6]	[5:3]	[2:0]
CSIN Rd, Rs	1010	Rs	000	Rd	000
CCOS Rd, Rs	1010	Rs	000	Rd	001
CTAN Rd, Rs	1010	Rs	000	Rd	010
ISIN Rd, Rs	1010	Rs	000	Rd	011
ICOS Rd, Rs	1010	Rs	000	Rd	100
ITAN Rd, Rs	1010	Rs	000	Rd	101

[15:12] Coprocessor Opcode — [11:9] Source Register — [8:6] Unused — [5:3] Destination Register — [2:0] Trigonometric Function

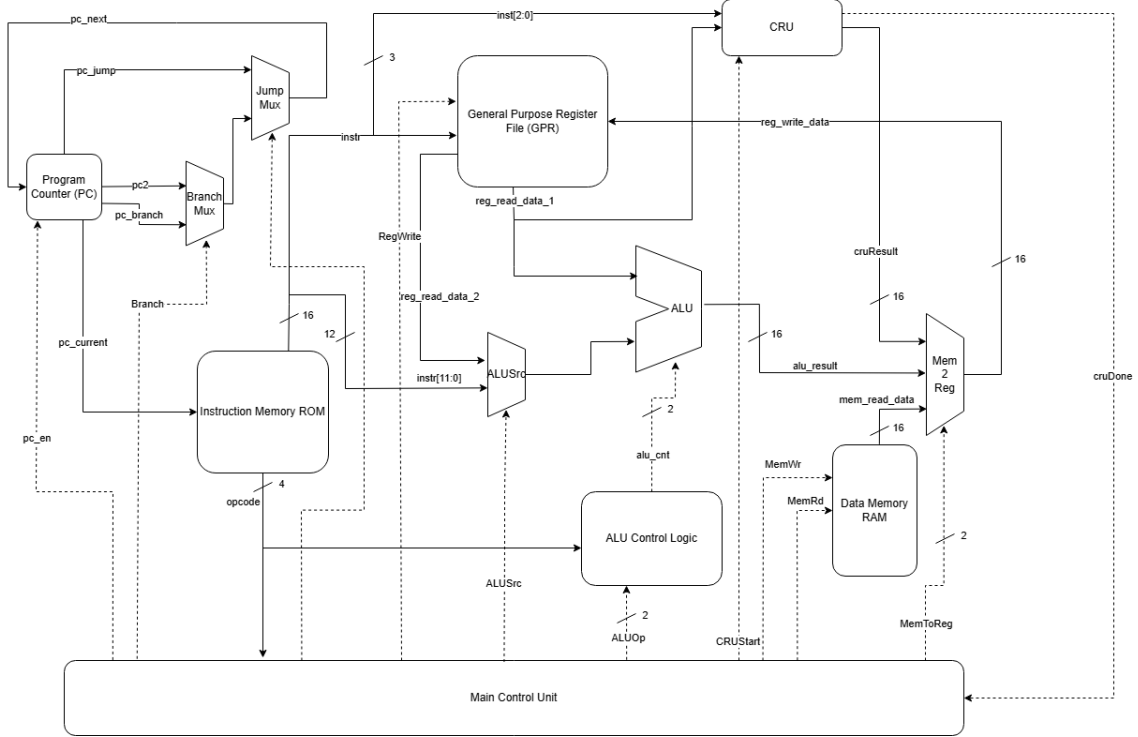


Figure 1: Modified StarCore-1 Datapath Architecture with CRU Integration

Block Diagram: StarCore-1 Processor + CORDIC CRU Co-Processor

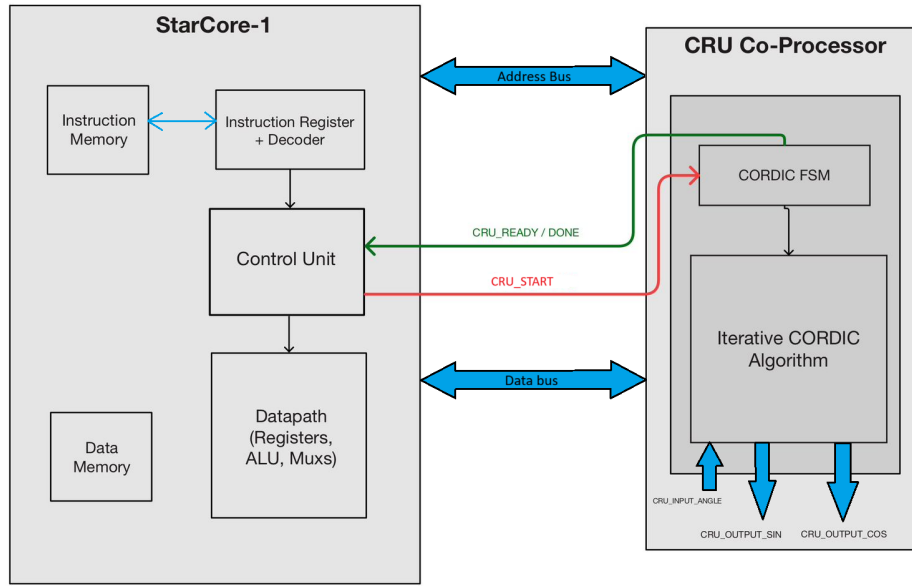


Figure 2: High-Level Block Diagram and FSM Interface of the Coordinate Rotation Unit

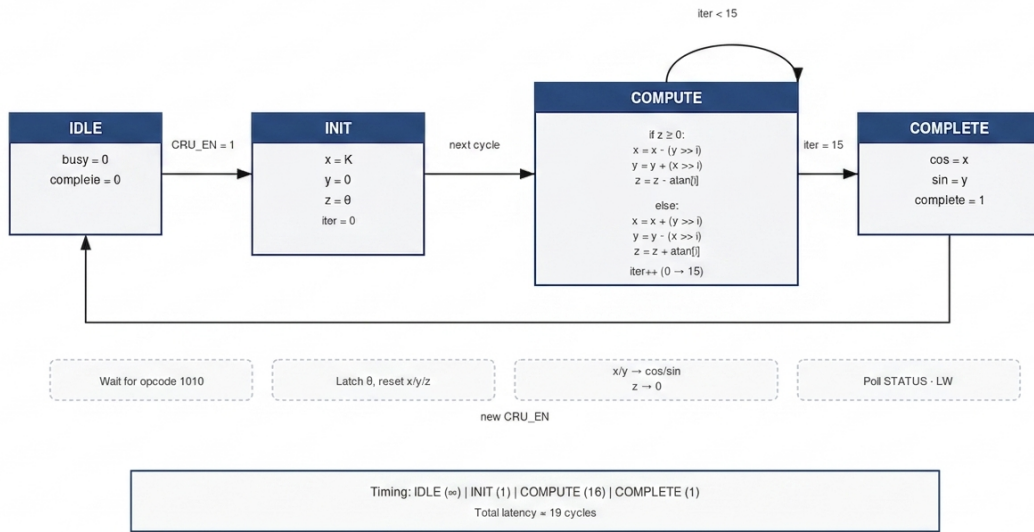


Figure 3: CRU Core 4-State FSM Statechart

CORDIC Validation Results

Table 2: CORDIC Validation Results (Matlab Reference Model)

Angle	cosExact	cosCORDIC	sinExact	sinCORDIC	Max error
0°	1.0000	1.0000	0.0000	0.0001	1 LSB
30°	0.8660	0.8660	0.5000	0.5000	1 LSB
45°	0.7071	0.7071	0.7071	0.7071	1 LSB
60°	0.5000	0.5000	0.8660	0.8659	1 LSB
90°	0.0000	0.0001	1.0000	0.9999	1 LSB
-45°	0.7071	0.7071	-0.7071	-0.7071	1 LSB

Appendix: Verilog Implementation and MATLAB Golden Measure

The following code snippets and MATLAB script were developed to simulate the iterative CORDIC algorithm. These tools were utilized to generate Q4.12 fixed-point reference data, serving as the 'Golden Measure' for Verilog testbench verification. This ensures that the hardware implementation of the core routing and CORDIC logic aligns with the mathematical model within defined precision limits.

CRU_Core.v

```
1 S_COMPUTE : begin
2 if (z_reg >= 16'sd0) begin
3 x_reg <= x_reg - (y_reg >>> iter);
4 y_reg <= y_reg + (x_reg >>> iter);
5 z_reg <= z_reg - $signed({1'b0, atan_lut[iter]});
6 end else begin
7 x_reg <= x_reg + (y_reg >>> iter);
8 y_reg <= y_reg - (x_reg >>> iter);
9 z_reg <= z_reg + $signed({1'b0, atan_lut[iter]});
10 end
11 iter <= iter + 4'd1;
12 if (x_reg == 16'sh8000 || y_reg == 16'sh8000) error <= 1'b1;
13 if (iter == 4'd15) state <= S_DONE;
14 end
```

ControlUnit.v

```
1 always @(*) begin
2 RegDst=0; ALUSrc=0; MemToReg=0; RegWrite=0;
3 MemRd=0; MemWr=0; Branch=0; ALUOp=2'b00; Jump=0;
4 CRU_EN = 1'b0;
5
6 case (opcode)
7 4'b1010 : begin
8 RegWrite = 1'b0;
9 MemWr    = 1'b0;
10 Branch   = 1'b0;
11 CRU_EN   = 1'b1;
12 end
13 endcase
14 end
```

StarCore1 CRU.v

```
1 module StarCore1_CRU (input clk, input reset);
2 wire [3:0] opcode;
3 wire CRU_EN; // new signal
4 wire RegDst, ALUSrc, MemToReg, RegWrite;
5 wire MemRd, MemWr, Branch, Jump;
6 wire [1:0] ALUOp;
7
8 ControlUnit ctrl (
9 .opcode(opcode), .RegDst(RegDst), .ALUSrc(ALUSrc),
10 .MemToReg(MemToReg), .RegWrite(RegWrite),
11 .MemRd(MemRd), .MemWr(MemWr), .Branch(Branch),
12 .ALUOp(ALUOp), .Jump(Jump),
13 .CRU_EN(CRU_EN) // new port
14 );
15
16 Datapath DU (
17 .clk(clk), .reset(reset),
18 .RegDst(RegDst), .ALUSrc(ALUSrc), .MemToReg(MemToReg),
19 .RegWrite(RegWrite), .MemRd(MemRd), .MemWr(MemWr),
20 .Branch(Branch), .ALUOp(ALUOp), .Jump(Jump),
21 .CRU_EN(CRU_EN), // new port
22 .opcode(opcode)
23 );
24 endmodule
```

Datapath.v

```
1 // Add to module port list:
2 output CRU_EN, // pass through from ControlUnit
```

```

3
4 // The memory bus connection changes from:
5 DataMemory dmem (
6     .clk(clk), .mem_address(alu_result),
7     .mem_data_in(reg_read_data2),
8     .MemRd(MemRd), .MemWr(MemWr),
9     .mem_data_out(mem_data_out)
10 );
11
12 // To (route through MMIO first):
13 wire [15:0] mmio_data_out;
14
15 MMIO_Controller mmio (
16     .clk(clk), .reset(reset),
17     .mem_address(alu_result),
18     .mem_data_in(reg_read_data2),
19     .MemRd(MemRd), .MemWr(MemWr),
20     .CRU_EN(CRU_EN),
21     .mem_data_out(mmio_data_out),
22     // CRU connections wired internally
23 );
24 // reg_write_data mux now uses mmio_data_out instead of mem_data_out

1 % Generates golden test vectors for the CRU Co-Processor (Q4.12 Format)
2
3 clear; clc;
4
5 iterations = 16;
6 fractional_bits = 12;
7 scale_factor = 2^fractional_bits;
8
9 target_angle_deg = 45;
10 target_angle_rad = deg2rad(target_angle_deg);
11
12 % Pre-compute the Arctangent Look-Up Table
13 atan_table_float = atan(2.^(-(0:iterations-1)));
14 atan_LUT = int16(round(atan_table_float * scale_factor));
15
16 % CORDIC Gain (K) Pre-computation
17 K_float = 1.0;
18 for i = 0:(iterations-1)
19     K_float = K_float * (1 / sqrt(1 + 2^(-2*i)));
20 end
21 K_quantized = int16(round(K_float * scale_factor));
22
23 fprintf('ROM Values for Verilog (Q4.12 format):\n');
24 fprintf('Initial X (K): %d (Hex: %s)\n', K_quantized, dec2hex(typecast(K_quantized, 'uint16')));
25 fprintf('LUT[0] to LUT[3]: %d, %d, %d, %d...\n\n', atan_LUT(1:4));
26
27 % Initialise hardware registers
28 x = int16(K_quantized);
29 y = int16(0);
30 z = int16(round(target_angle_rad * scale_factor));
31
32 % CORDIC iteration loop (Simulating the Verilog FSM)
33 fprintf('Iter\t X (Int)\t Y (Int)\t Z (Int)\n');
34
35 for i = 0:(iterations-1)
36
37     if z >= 0
38         d = 1;
39     else
40         d = -1;
41     end
42
43     % Store current state
44     x_curr = x;
45     y_curr = y;
46
47     % Emulate Verilog Arithmetic Right Shift (>>>)
48     x_shift = bitsra(x_curr, i);

```

```

49     y_shift = bitsra(y_curr, i);
50
51     % Emulate Adders/Subtractors
52     if d == 1
53         x = x_curr - y_shift;
54         y = y_curr + x_shift;
55         z = z - atan_LUT(i+1);
56     else
57         x = x_curr + y_shift;
58         y = y_curr - x_shift;
59         z = z + atan_LUT(i+1);
60     end
61
62     fprintf('%d\t %d\t\t %d\t\t %d\n', i, x, y, z);
63 end

```

Listing 1: CORDIC Golden Measure Generator (Q4.12 Format)